Machine Learning Notes

Andrew Valentini

September - December 2026

Contents

1 Linear Regression **2**

2 Cross-Validation **4**

3 Bayesian Optimization **6**

4 Linear Classification Models **12**
4.1 Naive Bayes Classifier 12
4.2 Gaussian Mixture Models 13

5 Classification **16**
5.1 Cluster Validity Metrics 16
5.2 K-Means & Fuzzy C-Means Clustering 17
5.3 Fisher's Linear Discriminant Analysis (LDA) 20
5.4 The Perceptron 24
5.5 Logistic Regression 27
5.6 Support Vector Machines (SVM) 29

6 Exercises **31**
6.1 Discriminant Function Derivation 31
6.2 Converting an Observed Data Likelihood into an Objective Function Expression 32
6.3 Expectation Maximization Algorithm Practice 33

7 To Know Cold **34**

1 Linear Regression

Want: a model to learn the relationship between data point $\{x_i\}$ and target parameters $\{t_ij\}$, where t_{ij} represents the jth target parameter corresponding to the ith data point (your data could be described by any number of parameters which you want a model to learn).

Example: x_i could be a person with features $x_i = \{\text{height, weight, age, sex, etc.}\}$. This representation of the data points x_i is called the feature space $\phi(x_i)$. In this example, target parameters t_{ij} could be a person's probability of cancer, life expectancy, etc. Each of these targets could be captured separately by $t_{i1}, t_{i2}, \ldots, t_{iN}$ for N possible targets you wish to map the observed data onto with a learned model.

Combined, $\{(x_i, t_{ij})\}_{i,j=1}^{N,M}$ is called the **feature space** for N distinct samples in your data and M distinct target parameters. With N samples and M desired features, the total feature space can be written as an $N \times M$ space.

$$\Phi = \begin{pmatrix} \phi(x_i)^T \\ \vdots \\ \phi(x_N)^T \end{pmatrix} \tag{1}$$

For example, if $\phi(x_i) = x_i$, then

$$\Phi = \begin{pmatrix} x_{11} & x_{12} & \ldots & x_{1M} \\ x_{21} & x_{22} & \ldots & x_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \ldots & x_{NM} \end{pmatrix} \tag{2}$$

Linear Regression assumes the output of the model is a linear combination of the input data

$$y(x) = \sum_{i=1}^{N} w_i x^i \tag{3}$$

The linear regression task can also be written as a matrix product

$$\boldsymbol{y} = X\boldsymbol{w} \tag{4}$$

Since $\boldsymbol{w}$ are free parameters and we wish to minimize our model's error in predicting the traget values t_i, we define an objective function,

$$J(\boldsymbol{w}) = \sum_{i=1}^{N} \frac{(t_i - y(x_i))^2}{N}, \tag{5}$$

a function which we seek to minimize to find the optimal set of $\boldsymbol{w}$ to best align the model with the data. Note that we assumed only one target value in this expression for simplicity.

To find the optimal $\boldsymbol{w}$ for a given linear regression task, we take

$$\frac{\partial J}{\partial W} = 0 \quad \text{for} \quad J(W) = \frac{1}{N}(T - XW)^T(T - XW)$$

Expanding $J(W)$:

$$J(W) = \frac{1}{N}\left(T^T T - T^T XW - W^T X^T T + W^T X^T XW\right)$$

Taking the partial derivative with respect to W:

$$\frac{\partial J}{\partial W} = \frac{1}{N}\left(-2X^T T + 2X^T XW\right)$$

Setting this to zero:

$$-2X^T T + 2X^T XW = 0$$

$$\implies X^T XW = X^T T$$

$$\implies \boxed{W^* = (X^T X)^{-1} X^T T}$$

To prevent overfitting, we can also add a **regularizer** term, which modifies the form of the objective function $J(\boldsymbol{w})$. One regularizer is called the Ridge regularizer. Adding this to the objective function looks like:

$$J(w) = \frac{1}{N}|T - XW|^2 + \frac{\lambda}{N}|W|^2,$$

which results in a form of the optimal W^* values written like

$$W^* = (X^T X + \lambda \mathbb{1}_n)^{-1} X^T T$$

obtained by the same process as before.

2 Cross-Validation

Goal: Find the optimal model hyperparameters for a particular regression task, which have so far been λ, the strength of regularization, and m, the model order.

To do this, the given data is typically given an 80/20 split into an *operational* and *testing* set, where the operational data is then further split into *training* and *validation*. The validation set is added to evaluate the model's performance for a given set of hyperparameters on unseen data. The optimal model hyperparameters are found by evaluating its performance under some error metric E and finding the set of parameters $X = \{\lambda, m\}$ that give $\frac{\partial E}{\partial X} = 0$ for the *validation* dataset. The error of the model computed for the training data will always decrease due to overfitting, whereas there will always be some minimum in the error of the validation's error, marking the parameter values where the model has begun to overfit on the training data. With the optimal hyperparameter values chosen, the overall performance of the model is then evaluated on the testing set.

Common error metrics for regression:

1. Mean-squared error:
$$\sum_{i=1}^{N} \frac{(\phi(x_i) - t_i)^2}{N} \tag{6}$$

2. Root mean-squared error:
$$\sqrt{\sum_{i=1}^{N} \frac{(\phi(x_i) - t_i)^2}{N}} \tag{7}$$
which requires that the differences $\phi(x_i) - t_i$ be positive to keep the overall error positive.

3. Mean absolute error:
$$\left| \sum_{i=1}^{N} \frac{\phi(x_i) - t_i}{N} \right| \tag{8}$$
which is ideal when you want your error metric to be unaffected by large errors (MSE's value will increase quadratically as error increases).

Techniques for cross-validation:

1. $K-$fold cross validation

2. $K \times 2$ cross validation

3. Stratified cross validation

Example of 4−fold cross validation:

First split the operational data into 4 equal subsets of the data $S = S_1+S_2+S_3+S_4$, run a grid search over all combinations of possible parameter values (within a defined range) $m \times \lambda = \{1, \ldots, 10\} \times \{0, .001, .01, 1\}$, evaluate an error metric over the model's performance for the training and validation data over each subset, and choose the set of parameters

Drawn out:

For $\lambda_j, m_j \in m \times \lambda = \{1, \ldots, 10\} \times \{0, .001, .01, 1\}$, evaluate

1. Training set $\{S_1, S_2, S_3\} \implies W^{(1)} \implies$ MSE of $x_t^{(1)}$
   Validation set $\{S_4\}$ under $W^{(1)} \implies$ MSE of $x_v^{(1)}$

2. Training set $\{S_1, S_2, S_4\} \implies W^{(2)} \implies$ MSE of $x_t^{(2)}$
   Validation set $\{S_3\}$ under $W^{(2)} \implies$ MSE of $x_v^{(2)}$

3. Training set $\{S_1, S_3, S_4\} \implies W^{(3)} \implies$ MSE of $x_t^{(3)}$
   Validation set $\{S_2\}$ under $W^{(3)} \implies$ MSE of $x_v^{(3)}$

4. Training set $\{S_2, S_3, S_4\} \implies W^{(4)} \implies$ MSE of $x_t^{(4)}$
   Validation set $\{S_1\}$ under $W^{(4)} \implies$ MSE of $x_v^{(4)}$

The total MSE performance of the model under a particular set of hyperparameters $\lambda_j, m_j \in \lambda \times m$ is the average of every fold's MSE $x_t^{(i)}$, $\sum_{i=1}^{4} x_t^{(i)}/4$ under these hyperparameter values. We also take the variance of the validation MSE values $x_v^{(i)}$ for each set of hyperparameters to determine how subject the model is to overfitting under them; $\sqrt{\sum_{i=1}^{4}\left(x_t^{(i)}\right)^2/4}$,since a model with a strong average MSE performance across all folds with a large variance in the MSE values would indicate the model greatly depends on which subset of data it is trained on (for the particular set of hyperparmeters $\{\lambda_j, m_j\}$).

The desired set of hyperparmeters $\{\lambda_j, m_j\}$ is therefore that which has a low average MSE across all folds *and* and low variance across these folds.

3 Bayesian Optimization

Bayesian optimization offers another method for finding ideal weight values. Again consider the goal of finding the set of W which minimize the objective function $J(W)$. Mathematically, the goal is again to find

$$\arg\min_W \frac{1}{N}\sum_{i=1}(t_i - y_i)^2 \tag{9}$$

for an objective function governed by the mean squared error. First note that finding W that minimizes the objective function is the same thing as finding W that maximizes the negative of the objective function:

$$\arg\min_x f(x) = \arg\min_x(-f(x)). \tag{10}$$

If we exponentiate this and multiply by $\frac{1}{\sqrt{2\pi}}$, we will get a form that looks like a Gaussian

$$\begin{aligned} &\arg\max_W \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{N}\sum_{i=1}^{N}(t_i - y_i)^2\right) \\ &= \arg\max_W \frac{1}{\sqrt{2\pi}} \prod_{i=1}^{N} \exp\left(-\frac{1}{N}(t_i - y_i)^2\right) \\ &= \arg\max_W \prod_{i=1}^{N} \mathcal{N}(t_i|y_i(W), 1) \end{aligned} \tag{11}$$

where we have used the notation $\mathcal{N}(x|\mu, \sigma^2)$, where x is the random variable we are sampling on the Gaussian distribution of mean μ and variance σ^2. Note that optimizing this expression will still result in the $W^* = (X^TX)^{-1}X^TT$ form we found earlier; this is just a different way of getting there.

If we instead let our objective function be described by the mean absolute error, we find

$$\begin{aligned} &\arg\max_W \prod_{i=1}^{N} \exp\left(-\frac{1}{2}|t_i - y_i|\right) \\ &\equiv \arg\max_W \mathcal{L}(t|y, 1), \end{aligned} \tag{12}$$

which is the exact form of a random variable t_i being drawn from the **Laplacian function** $\mathcal{L}(\mu, b)$, whose probability density function looks like

$$f_X(b) = \frac{1}{2b} \exp\left(\frac{-|x-\mu|}{b}\right). \tag{13}$$

Notice under this expression of weight optimization that when using the mean absolute error objective function, t values closer to the mean μ are more likely to be chosen than when using the mean squared error objective function since the Laplacian function is more highly peaked near μ than a Gaussian distribution.

We have shown that we can instead think about minimizing objective functions as minimizing probability distributions

$$\arg\min_W J(W) \equiv \arg\min_W P(t|W), \tag{14}$$

which is called the Bayesian interpretation of the objective function.

Extending this new expression of the objective function to include a regularizer term:

$$J(W) = \frac{1}{2}\sum_{i=1}^{N}(t_i - y_i)^2 + \frac{\lambda}{N}\sum_{j=1}^{M} w_j^2 \tag{15}$$

$$\begin{aligned}
\implies & \arg\max_W \exp\left(-\frac{1}{N}\sum_i (t_i - y_i)^2 + \frac{\lambda}{N}\right) \\
& \equiv \arg\max_W \prod_i \mathcal{N}(t_i|y_i, 1) \prod_j \exp\left(\frac{\lambda}{N} w_j^2\right) \\
& = \arg\max_W \prod_i \mathcal{N}(t_i|y_i, 1) \prod_j \mathcal{N}(w_j|0, \frac{1}{\lambda}) \\
& \equiv \arg\max_W P(t|W)P(W)
\end{aligned} \tag{16}$$

where we have used $P(W)$ to define the **prior** distribution on the weights and $P(t|W)$ is the likelihood, which is the distribution around t values given a set of weights W. Notice that, as expected, as λ increases, the variance of the prior increases, making the prior dominate the normal distribution which t_i's are being sampled from, meaning the regularizer term dominates the expression for finding the optimal W values.

We have thus further expressed the optimization problem as

$$\arg\min_W J(W) \equiv \arg\min_W P(t|W)P(W) \tag{17}$$

and we can use Bayes' theorem to say

$$\arg\min_W P(t|W)P(W) = P(t)\arg\min_W P(W|t) \tag{18}$$

$$\implies \arg\min_W P(W|t) = \frac{\arg\min_W P(t|W)P(W)}{P(t)} \tag{19}$$

which is called the **maximum a posteriori algorithm**. This is an especially important technique for finding optimal W values in scenarios when you have limited data and therefore cannot learn trends from the data alone (introducing the prior gets rid of this need).

Example: Consider a case of a limited dataset of coin flips $\{x_i\}_{i=1}^N$ where $x_i = \{1, 1, 1, 0, 0, 0\}$ for example. Say we want to determine which model generated these observations. We would expect this to look like a simple Bernouli distribution for a fair coin.

First define the probabilities of observing 1 (heads) and 0 (tails) as

$$P(x = 1|\mu) \equiv \mu \;\&\; P(x = 0|\mu) \equiv 1 - \mu. \tag{20}$$

The data likelihood for any observation x is then given as

$$P(x|\mu) = \mu^x(1 - \mu)^{1-x} \tag{21}$$

We know for a fair coin that $\mu \equiv 1 - \mu = .5$, but let us assume μ to be a free parameter to be learned from the maximum likelihood estimation method.

For the maximum likelihood estimation algorithm, we first define a total data observed likelihood function

$$\begin{aligned}\mathcal{L}^0 &= \prod_{i=1}^N P(x_i|\mu) = \prod_{i=1}^N \mu^{x_i}(1 - \mu)^{1-x_i} \\ &= \mu^{\sum_i x_i}(1 - \mu)^{\sum_i (1-x_i)} = \mu^{\sum_i x_i}(1 - \mu)^{N-\sum_i x_i}\end{aligned} \tag{22}$$

Converting this to a log-likelihood to make computation easier:

$$\begin{aligned}\ln \mathcal{L}^0 &= \ln\left(\mu^{\sum_i x_i}\right) + \ln\left((1 - \mu)^{N-\sum_i x_i}\right) \\ &= \sum_i x_i \ln(\mu) + \left(N - \sum_i x_i\right)\ln(1 - \mu)\end{aligned} \tag{23}$$

Finding the optimal μ value:

$$\frac{\partial \ln \mathcal{L}^0}{\partial \mu} = \sum_i \frac{x_i}{\mu} - \frac{(N - \sum_i x_i)}{1 - \mu} = 0 \tag{24}$$

$$\implies \mu_{\text{MLE}} = \frac{\sum_i x_i}{N} \tag{25}$$

Now using the maximum a posteriori algorithm to find the optimal μ value, we first need a prior distribution. Choose

$$\mu \sim \text{Beta}(a, b) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \mu^{a-1}(1-\mu)^{b-1} \tag{26}$$

where $a, b > 0$.

Setting up the observed likelihood for this MAP algorithm:

$$\mathcal{L}^0 = \left(\prod_{i=1}^{N} P(x_i|\mu) \right) \left(\prod_{i=j}^{M} P(\mu_j) \right) \tag{27}$$

where since we only have one parameter to optimize, $M = 1$. Using the same steps as before on the product of terms with sums in the exponential terms for $P(x_i|\mu)$, we get

$$\mathcal{L}^0 = \mu^{\sum_i x_i}(1-\mu)^{N - \sum_i x_i} \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \mu^{a-1}(1-\mu)^{b-1} \tag{28}$$

Since the term with the Gamma functions is just a constant, we can write

$$\mathcal{L}^0 = C\mu^{\sum_i x_i + a - 1}(1-\mu)^{N - \sum_i x_i + b - 1} \tag{29}$$

which is just the form of another Beta distribution with its constants reparameterized:

$$a' = \sum_i x_i + a \;\&\; b' = N - \sum_i x_i + b \tag{30}$$

giving us

$$\mathcal{L}^0 = C\mu^{a'-1}(1-\mu)^{b'-1}. \tag{31}$$

Note that whenever a posterior $P(y|t)$ is proportional to the prior, we call this a **conjugate prior relationship**.

$$\implies \ln \mathcal{L}^0 = \ln C + (a' - 1)\ln(\mu) + (b' - 1)\ln(1 - \mu) \tag{32}$$

$$\begin{aligned} \implies \frac{\partial \mathcal{L}^0}{\partial \mu} &= \frac{a' - 1}{\mu} + \frac{b' - 1}{1 - \mu} = 0 \\ \implies \mu_{\text{MAP}} &= \frac{\sum_i x_i + a - 1}{N + a + b - 2} \end{aligned} \tag{33}$$

This μ_{MAP} value is also data-driven, but it also now encodes additional information from the prior (Beta) distribution on a, b. Notice that if there was no prior information, the form of μ_{MAP} would reduce to μ_{MLE}.

We now wish to explicitly show that representing the objective function as a probability distribution in the Bayesian interpretation yields the same result for the optimal parameters W^* that we found from linear regression. Formally, we will construct and then minimize an objective function of a general Gaussian distribution and show this reduces to the solution $W^* = (X^T X)^{-1} X^T T$ found from linear regression.

Writing random variable vector $X = [x_1, \ldots, x^k]$ being sampled from a general Gaussian distribution with mean μ and variance Σ

$$\mathcal{N}(X|\mu, \Sigma) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(X - \mu)^T \Sigma^{-1}(X - \mu)\right) \tag{34}$$

where Σ is the **covariance matrix**, with terms

$$\Sigma_{ij} = \text{cov}[X_i, X_j] := \mathbb{E}[(X_i - \mathbb{E}[X_i])(X_j - \mathbb{E}[X_j])] \tag{35}$$

which physically measures the joint variability of two variables X_i and X_j.

Notice that if $X_i = X_j$,

$$\begin{aligned} \Sigma_{ii} &= \mathbb{E}\big[(X_i - \mathbb{E}[X_i])^2\big] = \mathbb{E}\big[X_i^2 - 2X_i\mathbb{E}[X_i]) + \mathbb{E}[X_i]^2\big] \\ &= \mathbb{E}\big[X_i^2\big] - 2\mathbb{E}[X_i\mathbb{E}[X_i]] + \mathbb{E}\big[\mathbb{E}[X_i]^2\big] \\ &= \mathbb{E}\big[X_i^2\big] - 2\mathbb{E}[X_i]\,\mathbb{E}[X_i] + \mathbb{E}[X_i]^2 \\ &= \mathbb{E}\big[X_i^2\big] - \mathbb{E}[X_i]^2 \equiv \sigma_i^2. \end{aligned} \tag{36}$$

Setting $\Sigma = 1$ for our case of a Gaussian with unity variance, we get

$$\ln \mathcal{N}(t|y, 1) = \frac{1}{2}\ln\left(\frac{1}{\sqrt{2\pi}}\right) - \frac{1}{2}|t - y|^2 \tag{37}$$

after applying the log.

Setting the partial with respect to W equal to 0:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W} &= \frac{\partial}{\partial W}\left(\frac{1}{2}\ln\left(\frac{1}{\sqrt{2\pi}}\right) - \frac{1}{2}|t - XW|^2\right) = 0 \\ &\implies W^* = (X^T X)^{-1} X^T t \end{aligned} \tag{38}$$

which is the same solution we found with normal linear regression, showing that representing objective functions as probability distributions to be maximized under the Bayesian interpretation, $\arg\max_W \mathcal{N}(t|y, 1)$, is equivalent to the methodology developed in linear regression.

This connection also allows us to find optimal W values very quickly by using information developed in linear regression. If we are instead maximizing the product of two Gaussians:

$$\arg\max_W \mathcal{N}(t|y, 1)\mathcal{N}(W|0, 1/\lambda), \tag{39}$$

we can now immediately recognize the solution to this to be equivalent to the linear regression task with the ridge regularizer:

$$W^* = (X^T X + \lambda \mathbb{1})^{-1} X^T t \tag{40}$$

4 Linear Classification Models

4.1 Naive Bayes Classifier

Goal: Assign data to distinct classes. In other words, find the probability of a data point x_i belonging to a class C_k.

We do this by assigning probability distributions on the observed data for points to belong to a given class. Usually these class probability distributions are just Gaussians:

$$P(x_i|C_k) = \mathcal{N}(x_i|\mu_k, \Sigma_k) \tag{41}$$

Where $P(x_i|C_k)$ is the likelihood; the probability that a random data point x_i would be pulled from class C_k. If we want to find the probability of each class existing in the total dataset, we use Bayes Theorem to convert the likelihood into a posterior:

$$P(C_k|x_i) = \frac{P(x_i|C_k)P(C_k)}{P(x_i)} \tag{42}$$

where $P(C_k)$, the prior, determines how frequently instances from class C_k are drawn from the full dataset (how present C_k is in the dataset). $P(x_i)$ is the normalization factor to keep $P(C_k|x_i) \in [0, 1]$, which is a sum over classes $\sum_j P(x_i|C_k)P(C_k)$.

The "naive" assumption here comes from assuming that the data features features are independent from each other. In other words:

$$P(X|C_k) = \prod_{i=1}^{N} P(x_i|C_k) \tag{43}$$

In this formatting of the classification problem, we have introduced two new hyperparameters to be chosen, which are the forms of the likelihood and priors. Using this method, we determine a datapoint x_i to belong to class C_k for which the posterior probability is maximized:

$$P(C_1|x_i) \lesseqgtr_{x_i \in C_1}^{x_i \in C_2} P(C_2|x_i) \tag{44}$$

and in general for k classes:

$$x_i \in C_k \quad \text{if} \quad k = \arg\max P(C_k|x_i) \tag{45}$$

We can also use this Bayesian design of the classification problem to define a closed form **discriminant** function $g(\boldsymbol{x})$ that clearly distinguishes the regions where class membership is defined in the feature space.

We motivate the construction of $g(\boldsymbol{x})$ by first considering a two class classification problem, where $x_i \in C_1$ which implies

$$\begin{aligned} P(C_1|x_i) > P(C_2|x_i) \implies \ln\left(\frac{P(C_1|x_i)}{P(C_2|x_i)}\right) > 0 \\ \implies g(\boldsymbol{x}) \equiv \ln\left(\frac{P(C_1|x_i)}{P(C_2|x_i)}\right) \end{aligned} \tag{46}$$

$$\implies g(\boldsymbol{x}) = \ln\big(P(C_1|x_i)\big) - \ln\big(P(C_2|x_i)\big) \equiv g_1(\boldsymbol{x}) - g_2(\boldsymbol{x}) > 0 \tag{47}$$

meaning that regions in the feature space where the above expression is greater than zero is where our model is classifying data points to belong to class C_1. All regions in the data where the discriminant function $g(\boldsymbol{x}) < 0$ is then the region in the data where our model is defining all points with C_2 membership to exist. Independently, $g_1(\boldsymbol{x})$ defines the space in the data where class membership is to class C_1.

4.2 Gaussian Mixture Models

This is an unsupervised classification method which is especially useful for the existence of multiple clusters in data and when the exact class descriptions may not be known. In this model, the likelihood becomes a weighted sum of K Gaussians:

$$P(x_i|\theta) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k) \tag{48}$$

where the π_k are the prior probabilities that a sample belongs to class k, subject to the constraints $0 \leq \pi_k \leq 1$ and $\sum_k \pi_k = 1$.

Just as before, to find the optimal parameter values $\theta_k = \{\pi_k, \mu_k, \Sigma_k\}$ for this classification problem, the total data likelihood is constructed over all samples;

$$\mathcal{L} = \prod_{i=1}^{N} \sum_{k=1}^{K} \pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k), \tag{49}$$

where the usual process would then be to take $\frac{\partial \ln \mathcal{L}}{\partial \theta}\big|_{\theta^*} = 0$. This has now become an intractable problem, however, since we are taking the product of

a sum. The method for dealing with this is called the **Expectation Maximization Algorithm (EM)**. This algorithm introduces a latent variable z_{ik} which represents the belongingness of datapoint i to class k (a one-hot vector where the only non-zero entry is for the class k that sample i actually belongs to). We can then write the likelihood as

$$\mathcal{L} = \prod_{i=1}^{N}\prod_{k=1}^{K}\Big(\pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k)\Big)^{z_{ik}} \tag{50}$$

since when a sample i belongs to class k, the only remaining term in the products will be $\pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k)$. This is equivalent to a discrete Dirac delta function, but is placed in the exponent so that the log is easier to take.

When the log of this expression is taken, we get

$$\ln \mathcal{L} = \sum_{i=1}^{N}\sum_{k=1}^{K} z_{ik}\Big(\ln(\pi_k) + \ln\Big(\mathcal{N}(x_i|\mu_k, \Sigma_k)\Big)\Big) \tag{51}$$

This is now the quantity we want to use to find the optimal $\theta_i = \{\pi_i, \mu_i, \Sigma_i\}$, but z_{ik} is still an unknown latent variable, so we can average over its contributions using an expectation value of the previous expression, giving us the EM objective function:

$$Q(\theta, \theta^{(t)}) = \mathbb{E}_{z_i}\left[\sum_{i=1}^{N}\sum_{k=1}^{K} z_{ik}\Big(\ln(\pi_k) + \ln\Big(\mathcal{N}(x_i|\mu_k, \Sigma_k)\Big)\Big)\middle| X, \theta^{(t)}\right] \tag{52}$$

(given the full dataset X and initial parameter values $\theta^{(t)}$, we update the parameters according to this algorithm).

Since the expectation value only acts on terms that contain dependence on z_{ik}, we get

$$Q(\theta, \theta^{(t)}) = \sum_{i=1}^{N}\sum_{k=1}^{K} \mathbb{E}\left[z_{ik}\middle| X, \theta^{(t)}\right]\Big(\ln(\pi_k) + \ln\Big(\mathcal{N}(x_i|\mu_k, \Sigma_k)\Big)\Big) \tag{53}$$

To further express the expectation value, consider a two-class problem where sample $x_i \in C_1$, meaning $\mathbb{E}\big[z_{ik}\big|x_i, \theta^{(t)}\big] = 1 \cdot P(z_{ik} = 1|x_i, \theta) + 0 \cdot P(z_{ik} =$

$0|x_i, \theta) = P(z_{ik} = 1|x_i, \theta)$, which we can further express using Bayes theorem:

$$\begin{aligned} P(z_{ik} = 1|x_i, \theta) &= \frac{P(x_i|z_{ik} = 1, \theta)P(z_{ik} = 1)}{P(x_i)} \\ &= \boxed{\frac{\mathcal{N}(x_i|\mu_k^{(t)}, \Sigma_k^{(t)})\pi_k^{(t)}}{\sum_{j=1}^{K} \mathcal{N}(x_i|\mu_j^{(t)}, \Sigma_j^{(t)})\pi_j^{(t)}} \equiv \gamma_{ik}} \end{aligned} \tag{54}$$

We then have the following form of the objective function:

$$\boxed{Q(\theta, \theta^{(t)}) = \sum_{i=1}^{N} \sum_{k=1}^{K} \gamma_{ik} \Bigg(\ln(\pi_k) + \ln\Big(\mathcal{N}(x_i|\mu_k, \Sigma_k) \Big) \Bigg)} \tag{55}$$

5 Classification

5.1 Cluster Validity Metrics

Goal: want way of determining the optimal number of groups in unsupervised learning tasks.

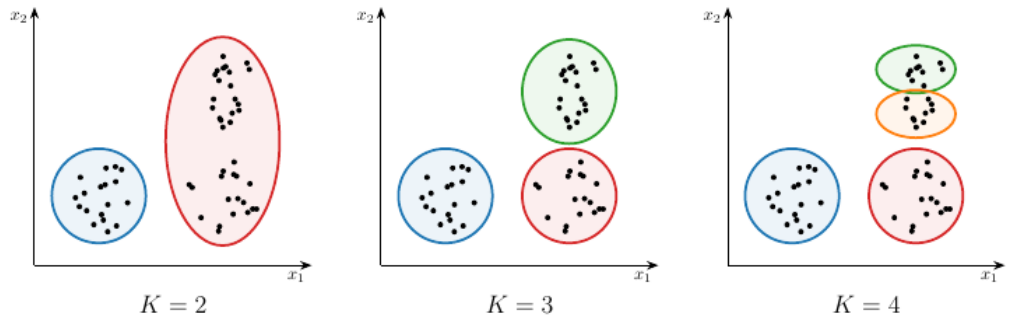


Figure 1: We see that $K = 3$ is optimal, but how do we get this to be discovered by a model in an unsupervised task?

We can define **internal criteria** on an unsupervised model, which requires that its learned groups be compact and separable. One example of an internal criterion is called the **silhouette index** $-1 \leq s \leq 1$, defined as

$$s = \frac{1}{N} \sum_{i=1}^{N} \frac{b_i - a_i}{\max(a_i, b_i)} \tag{56}$$

where N is the total number of data points being considered in the classification task, a_i is the average distance between a sample x_i and all other points in the same cluster, and b_i is the average distance between a sample and all other points in different clusters. From this definition, we see that a high s value is a good one, since it indicates that within-group points are close while out-of-group points are distinct. Notice that $s = 1$ is nonphysical, however, since it requires that $a_i = 0$, that there exists no distance between points in the same class, which corresponds to a scenario where each point is represented by its own cluster.

With an internal criteria like the silhouette index, we can then do a hyperparameter grid search to find the optimal value.

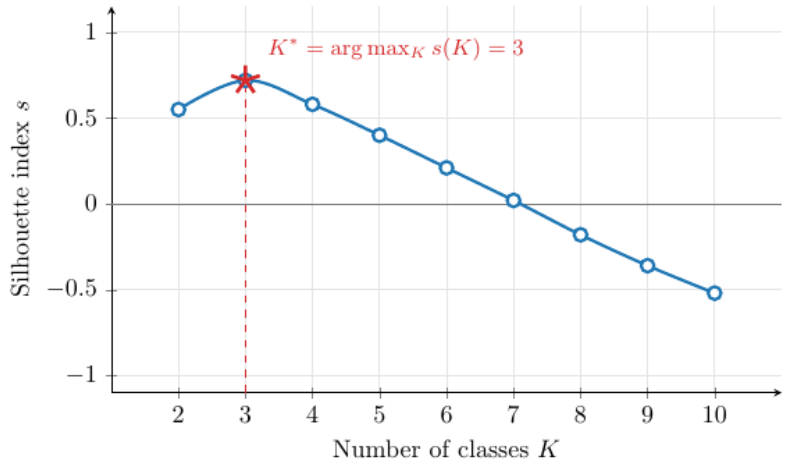


Figure 2: $K = 3$ would be chosen for this classification task since it is the value for which the silhouette index is maximized.

One can alternatively consider **external criteria** for determining the optimal number of classes, which requires the use of external information, such as ground truth labels or by comparing results between clustering algorithms. One such example is the **Rand index** $0 \leq R \leq 1$:

$$R = \frac{a+b}{a+b+c+d} \tag{57}$$

5.2 K-Means & Fuzzy C-Means Clustering

Once the hyperparameter K is chosen, either by internal or external criteria, one of the most common clustering algorithms, called K-means, can be used. K-means is carried out by initializing the position of *centroids* for each of the K clusters θ_k and then assigning class membership to a point x_i based on whichever centroid it is closest to. Once class membership is assigned for all data points, the positions of the centroids are then re-computed based on the positions of all of the points that were picked up in its respective class. Class membership is computed again based on these updated centroid positions, and this is done iteratively until convergence is reached.

Note that the convergence of the K-means algorithm depends heavily on the initialization of the centroids, meaning that you are not guaranteed to find the optimal solution. The iteration of the K-means algorithm is carried out

by the expectation maximization (EM) algorithm, where the E-step assigns class membership given θ_k and the M-step updates the positions of θ_k given the data. The objective function for the K-means EM optimization algorithm is given as

$$J(x_i, u_{ik}) = \sum_{i=1}^{N} \sum_{k=1}^{K} u_{ik} d(x_i|\theta_k) \tag{58}$$

where $u_{ik} \in \{0, 1\}$ is the defined class k membership of point i and $\sum_k u_{ik} = 1$ to capture the fact that a point *must* belong to some class k. $d(x_i|\theta_k)$ is the defined distance metric used to define how far each point x_i is from each cluster centroid θ_k. Notice that when $K = N$ (when each data point assumes its own class), $J = 0$, which is the trivial classification of the data.

Common distance metrics include

1. Mahalanobis distance:

$$d_M(x_i, \theta_k) = \sqrt{(x_i - \theta_k)\Sigma_k^{-1}(x_i - \theta_k)} \tag{59}$$

2. Euclidean distance:

$$d_E(x_i, \theta_k) = \sqrt{(x_i - \theta_k)(x_i - \theta_k)} \tag{60}$$

3. Manhattan distance

Note that in high-dimensional feature spaces, the concept of distance becomes an ill-defined metric to use since the distance between a sample point and its closest point and furthest point converge. This is called the **curse of dimensionality**.

In K-means clustering, the Euclidean metric is used, so one must scale the model's features so that a distance between spaces with unequal units can actually be drawn. Techniques for scaling a dataset's features before using K-means clustering include

1. Standardization/Normalization

$$\hat{f_i} = \frac{f_i - \mu_{f_i}}{\sigma_{f_i}} \tag{61}$$

   Notice that this converts the dimensions of each feature into a dimensionless z-score.

2. Min-Max Scalar:

$$\hat{f}_i = \frac{f_i - \min(f_i)}{\max(f_i) - \min(f_i)} \in [0, 1] \tag{62}$$

3. Robust min/max

$$\hat{f}_i = \frac{f_1 - f_{15}}{f_{85} - f_{95}} \tag{63}$$

In K-means, class membership, determined by u_{ij} was fixed ($u_{ij} \in \{0, 1\}$). Fuzzy C-means relaxes this strict membership assignment by allowing u_{ij} to vary continuously between 0 and 1 ($u_{ij} \in [0, 1]$). The objective function for fuzzy C-means looks similar to K-means:

$$J(w) = \sum_{c=1}^{C} \sum_{i=1}^{N} u_{ic}^m d_{ic}^2 \tag{64}$$

where I have switched $u_{ij} \to u_{ic}$ just to indicate that the class membership is now determined by the fuzzy C-means method. Notice we have additionally added the parameter m into the objective function, which represents the "fuzziness" of the clustering, where as m increases, the fuzziness of the classification increases, meaning that points will have higher membership values for multiple classes.

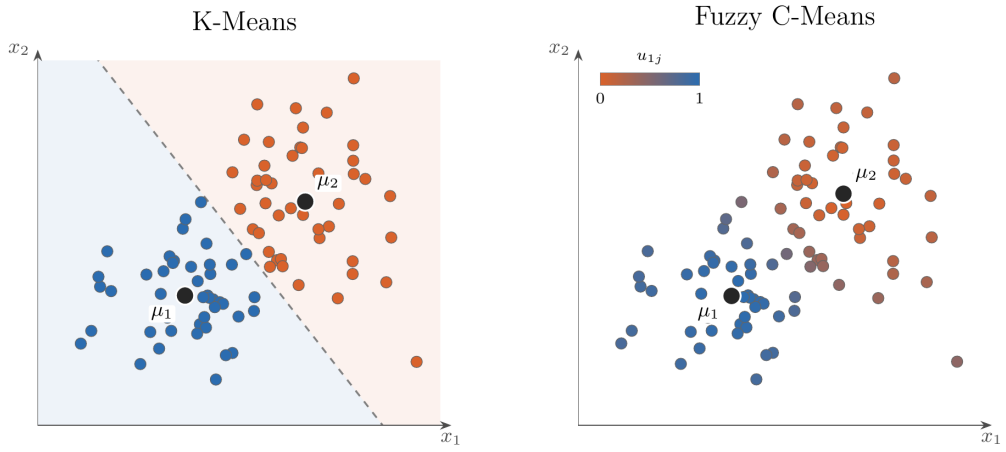


Figure 3: K-means assigns hard class membership with $u_{ij} \in \{0, 1\}$ where fuzzy C-means assigns soft membership with $u_{ij} \in [0, 1]$. FCM therefore includes a confidence level of class membership determined by its u_{ij} value.

Once the u_{ic} values are determined for all data points, class membership is then assigned using some threshold criteria $0 < \tau < 1$. Notice that as τ increases, class membership is determined by much tighter circles. If we want to clearly express the class membership as being determined by a circle in the data space with some radius, we first again state that class membership is determined by

$$u_{ic} = \frac{1}{1 + \left(\frac{d_{ic}}{\eta_c}\right)^{\frac{1}{m-1}}} > \tau \tag{65}$$

where η_c is the scale factor for each cluster (how far each cluster reaches), so we want to find an expression for how d_{ic} relates to η_c:

$$\begin{aligned} &\implies \left(\frac{d_{ic}}{\eta_c}\right)^{1/m-1} \leq \frac{1}{\tau} - 1 \\ &\implies d_{ic} \leq \eta_c \left(\frac{1-\tau}{\tau}\right)^{m-1} \end{aligned} \tag{66}$$

5.3 Fisher's Linear Discriminant Analysis (LDA)

Supervised learning algorithm for classification by finding the linear combination of features that best separates data into K classes.

Under the assumption that all classes can be represented as independent Gaussians, the LDA algorithm first initializes classes with $\{\mu_k, \Sigma_k\}$.

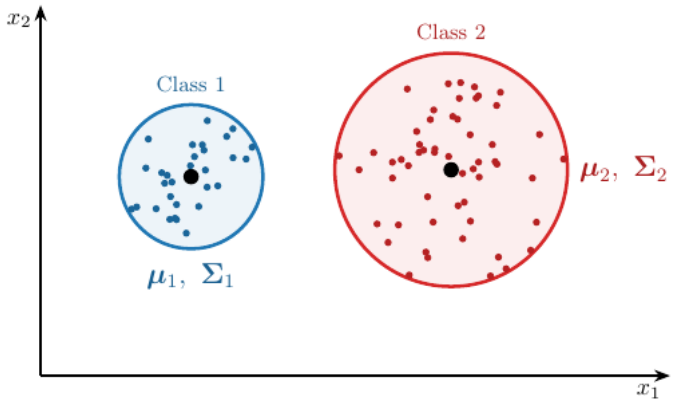


Figure 4: Defining classes under the assumption that all can be modeled by Gaussians $\mathcal{N}(x_i|\mu_k, \Sigma_k)$

LDA then identifies the line through the data space which makes the *projected* means m_k furthest apart (distance measured along the projection line) and the projected variances σ_i^2 of each class the most compact.

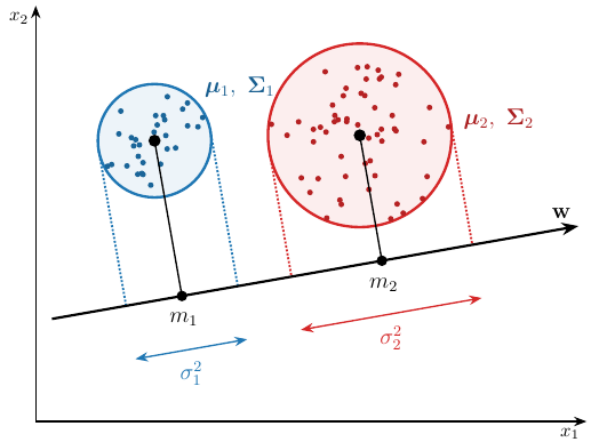
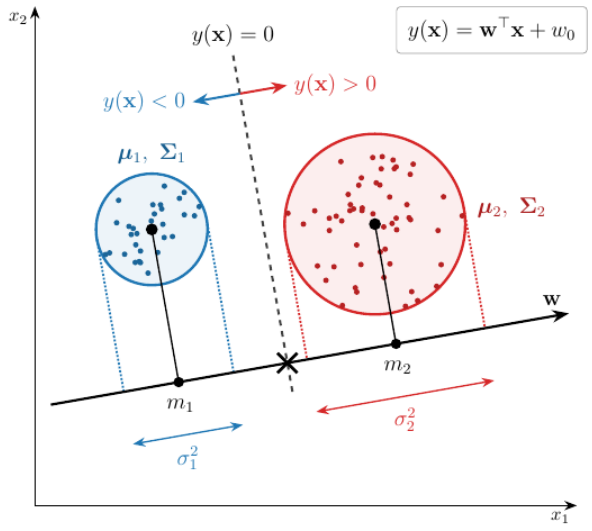


Figure 5: Projecting $\{\mu_i, \Sigma_i\}$ onto a line, whose direction is given by $\boldsymbol{w}$.

Once this optimal projection is drawn, the center of the projection line is determined by taking the center between all m_i's and drawing a line tangent to the projection line, making sure the tangent line passes through this center point, which is $(m_1 + m_2)/2$ for 2-class. This tangent line is the discriminant function $y(\boldsymbol{x})$



Just as in the Naive Bayes classifier, class membership for a point x_i is determined by whichever class gives the point a positive value for its discriminant function.

$$\begin{aligned} x_i \in C_1 \quad &\text{if} \quad y(x_i) > 0 \\ x_i \in C_{-1} \quad &\text{if} \quad y(x_i) < 0 \end{aligned} \tag{67}$$

where -1 has indicated "not class 1," which is clearly class 2 in a 2-class problem, but in the case of multiple classes, we have demonstrated that the membership of a point with a negative a discriminant value for all possible classes is undetermined.

The objective function of LDA for a 2-class problem is

$$J(w) = \frac{(m_1 - m_2)^2}{\sigma_1^2 + \sigma_2^2} \tag{68}$$

since this promotes the maximization of the difference between projected means while also promoting the minimization of the projected variances.

To write this explicitly in terms of W, we need to use the fact that $m_i = \boldsymbol{\mu}_i \cdot \boldsymbol{w}$. Projecting the differences in means:

$$(m_1 - m_2)^2 = \left(\frac{\boldsymbol{w}^T}{\|\boldsymbol{w}\|} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) \right) \cdot \left(\frac{\boldsymbol{w}^T}{\|\boldsymbol{w}\|} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) \right)^T \tag{69}$$

And if you define the between class scatter matrix $S_B = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) \cdot (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^T$, you get

$$(m_1 - m_2)^2 = \frac{\boldsymbol{w}^T S_B \boldsymbol{w}}{\|\boldsymbol{w}\|^2} \tag{70}$$

Now for the projected variance:

$$\sigma_k^2 = \frac{1}{N_k} \sum_{n \in C_k} (x_n - m_k)^2 \tag{71}$$

where N_i is the number of samples in class k. Taking $(x_n - m_k) \to \boldsymbol{w}^T \cdot (\boldsymbol{x}_n - \boldsymbol{\mu}_k)$:

$$\begin{aligned} \sigma_k^2 &= \frac{1}{N_k} \sum_{n \in C_k} \Big(\boldsymbol{w}^T \cdot (\boldsymbol{x}_n - \boldsymbol{\mu}_k) \Big) \cdot \Big(\boldsymbol{w}^T \cdot (\boldsymbol{x}_n - \boldsymbol{\mu}_k) \Big)^T \\ &= \boldsymbol{w}^T \left(\frac{1}{N_k} \sum_{n \in C_k} (\boldsymbol{x}_n - \boldsymbol{\mu}_k) \cdot (\boldsymbol{x}_n - \boldsymbol{\mu}_k) \right) \boldsymbol{w} \end{aligned} \tag{72}$$

and defining the within-class scatter matrix S_W, we get

$$\sigma_k^2 = \boldsymbol{w}^T S_W \boldsymbol{w} \tag{73}$$

With the full form of the objective function now explicitly written with a dependence on $\boldsymbol{w}$,

$$J(\boldsymbol{w}) = \frac{\boldsymbol{w}^T S_B \boldsymbol{w}}{\boldsymbol{w}^T S_W \boldsymbol{w}}, \tag{74}$$

we can find its optimal solution using the quotient rule, which eventually gives:

$$\frac{\partial J}{\partial \boldsymbol{w}} = \frac{S_B \boldsymbol{w}}{\boldsymbol{w}^T S_W \boldsymbol{w}} - \frac{\boldsymbol{w}^T S_B \boldsymbol{w}}{\boldsymbol{w}^T S_W \boldsymbol{w}} \frac{S_W \boldsymbol{w}}{\boldsymbol{w}^T S_W \boldsymbol{w}} = 0 \tag{75}$$

And if we define $\lambda = \frac{\boldsymbol{w}^T S_B \boldsymbol{w}}{\boldsymbol{w}^T S_W \boldsymbol{w}}$, we get an eigenvalue equation

$$\frac{S_B \boldsymbol{w}}{\boldsymbol{w}^T S_W \boldsymbol{w}} = \lambda \frac{S_W \boldsymbol{w}}{\boldsymbol{w}^T S_W \boldsymbol{w}} \implies (S_W^{-1} S_B) \boldsymbol{w} = \lambda \boldsymbol{w} \tag{76}$$

so $\boldsymbol{w}$ is an eigenvector of $S_W^{-1} S_B$.

Back to the point that LDA only prescribes membership iff $y_k(x_i) > 0$, there could be some undetermined region where all classes predict $y_k(x_i) < 0$. In this case, LDA assigns class membership to the class with the largest

discriminant value. But in doing this, the weights across all discriminant function definitions must be normalized for proper comparison.

Since for a single point x_i, the prediction of all discriminant functions

$$y_k(x_i) = w_k^T x_i + w_{0k} \tag{77}$$

must be accounted for, the LDA algorithm acts like a **perceptron**, which is a binary linear classifier which was introduced in the 1970s, which is the fundamental building block of a neural network.

In training with ground truth labels, the LDA objective function looks like

$$J(\boldsymbol{w}, \boldsymbol{w}_0) = \sum_{n \in M} t_n(\boldsymbol{w}^T \boldsymbol{x}_n + \boldsymbol{w}_0) \tag{78}$$

where M are all of the LDA model's misclassified points. This is then optimized used **gradient descent**:

$$\boldsymbol{w}^{(i+1)} = \boldsymbol{w}^{(i)} - \eta \frac{\partial J(\boldsymbol{w}^{(i)})}{\partial \boldsymbol{w}} \tag{79}$$

where η is the learning rate/step size.

Note that the max number of projections for a case of D dimensional data and K classes is $\max(D-1, C-1)$.

5.4 The Perceptron

Another linear classifier that learns a potentially high-dimensional line $y(\boldsymbol{x}) = \boldsymbol{w}^T \boldsymbol{x} + \boldsymbol{b}$ to separate classes in potentially high-dimensional spaces.

The perceptron differs from LDA in how it learns the $\boldsymbol{w}$ and $\boldsymbol{b}$ parameters in the discriminant function. The perceptron learns them through trial and error (either through stochastic or batch optimization... see below), while LDA computes the line in an analytical closed form expression from the projected class means and variances.

Note that even when the data is linearly separable, the perceptron algorithm does *not* converge to that of LDA, again due to the fact that the learning of $y(\boldsymbol{x})$ is different across algorithms. In fact, due to the stochastic nature of the perceptron's learning algorithm and its dependence on the random initialization of $\boldsymbol{w}^{(0)}$ and $\boldsymbol{b}^{(0)}$, using the perceptron to classify the same data multiple times may not even give the same result. The LDA algorithm, although it learns lines in an exact closed form, can lead to misclassification,

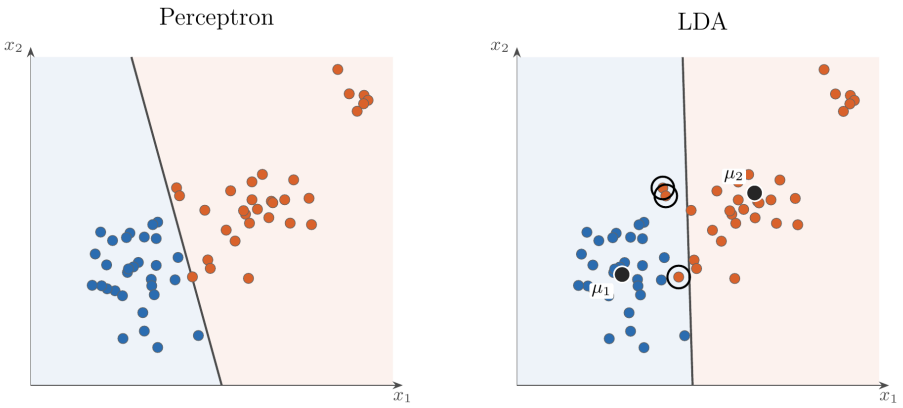


Figure 6: An example of how a perceptron classifies data versus LDA. Notice the perceptron's zero classification error versus LDA's misclassified points, circled in black.

even in the training set, if there are significant outliers to a class's projected mean and variance.

Back to the objective function for the perceptron:

$$J(\boldsymbol{w}, \boldsymbol{w}_0) = \sum_{n \in M} t_n(\boldsymbol{w}^T \boldsymbol{x}_n + \boldsymbol{w}_0) \tag{80}$$

For a 2 class problem, $t_n \in \{-1, 1\}$ where the sign of the t_n identifies the exact type of misclassification. Note that these misclassifications can be identified by the sign if the product of $t_n y(x_n)$; if $y(x_n) > 0$ (the perceptron's classification determines x_n to belong to C_1), and $x_n \in C_1$ in reality, then the sign of the product will be positive. The misclassifications comes from cases when either the model predicts $x_n \in C_1$ (+) when $x_n \in C_2$ in reality (-) or when the model predicts $x_n \in C_1$ (-) when $x_n \in C_2$ in reality (+). Note that for $K > 2$ classification tasks, t_n becomes a one-hot vector, where $t_n = [1, 0, 0]$ would indicate $x_n \in C_1$, for example.

We can optimize the perceptron's objective function stochastically (per-sample) or in batches of the training data. **Stochastic optimization** requires computing the gradient for each sample in the training data randomly chosen, so:

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \nabla J(\boldsymbol{w}^{(t)}) = \boldsymbol{w}^{(t)} - \eta \boldsymbol{x}_n t_n \tag{81}$$

where $J(\boldsymbol{w}^{(t)})$ is the objective function for *the single point* x_n. In **batch**

optimization, you construct the objective function over *all* of the training data, so

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \nabla \sum_{n=1}^{N} J_n(\boldsymbol{w}^{(t)}) \tag{82}$$

These differences imply that

1. For a single optimization step, stochastic learning is highly dependent upon the specific data point that is randomly chosen and the optimization path over time is therefore, as the name implies, highly erratic or stochastic. Stochastic learning is therefore computationally inexpensive for each update.

2. In batch learning, the individual weight updates take long to compute since they involve taking a potentially high-dimensional gradient over all the training data's contributions, and the optimization path that is traced out is therefore much smoother.

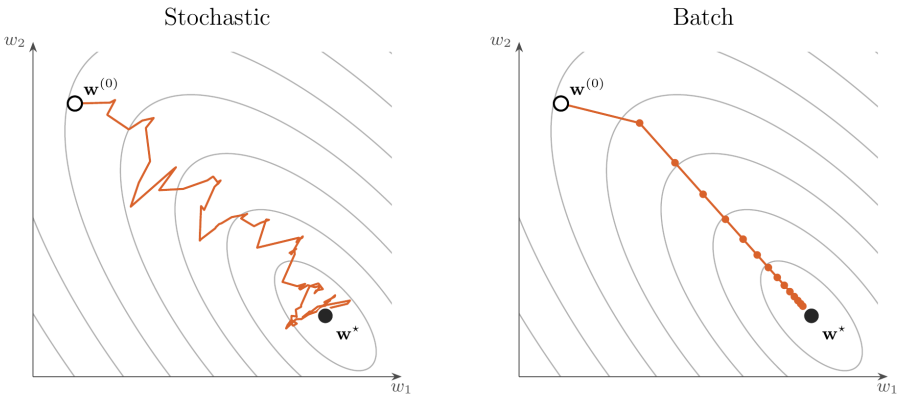


Figure 7: Demonstrating the difference between stochastic optimization and batch optimization.

An intermediate algorithm exists, which is even usually preferred, and involves splitting the full training dataset into **mini-batches**, where the subsets of the full training are randomly drawn and used in a given optimization step. This gives the benefit from stochastic learning in allowing the individual steps to be computed more quickly , while also being less erratic than the pure stochastic learning approach.

It is often computationally advantageous to dynamically update the learning rate η. Once can imagine a scenario in which the perceptron's objective function requires a great deal of optimization and a small enough learning rate would lead to the convergence time to the optimal solution being excessively large. One can similarly imagine a scenario in which too large of a learning rate is chosen, and a true minima is skipped over in the optimization steps.

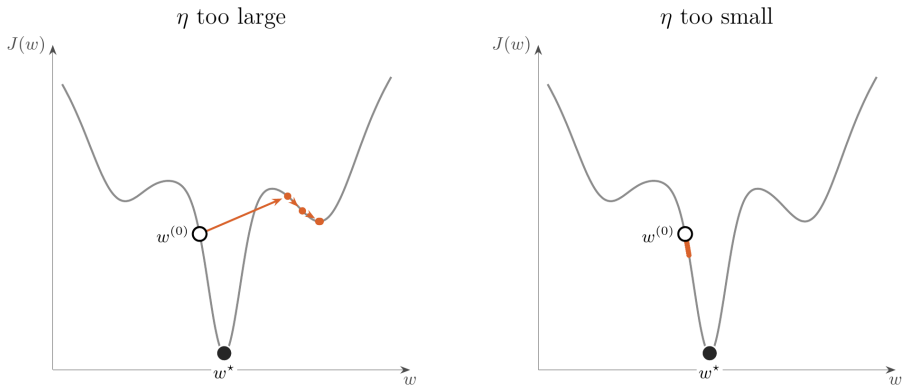


Figure 8: Demonstrating the dangers of a fixed learning rate. Too small of a learning rate could lead to unnecessarily long convergence time, while too large of a learning rate could lead to the optimization missing the minimum.

One way of combating these issues is to check the sign of the objective function's gradient before and after each step, because if a step jumps a minimum, the sign of the gradient will change between these two stages $\nabla J(\boldsymbol{w}^{(t+1)})\nabla J(\boldsymbol{w}^{(t)}) < 0$. If not sign change is detected over a step $t \to t+1$, one can update the learning rate by either $\eta' = \eta + \alpha$ or $\eta' = \beta\eta$.

5.5 Logistic Regression

One method of adding a confidence level to a classification's task; when you want to say a data point x_n belongs to class C_k with an $X\%$ chance, is through logistic regression. Instead of assigning $t_n \in \{0, 1\}$, or one-hot encoding for multi-dimensional classification, logistic regression maps objective function values between the range $[0, 1]$ by

$$y(\boldsymbol{x}_n) = \phi(\boldsymbol{w}^T \boldsymbol{x}_n + \boldsymbol{w}_0) \tag{83}$$

where

$$\phi(v) = \frac{1}{1+e^{-v}} \in [0,1]. \tag{84}$$

This means that if points are far from the logistic boundary, they are being said to belong to either class 0 or 1 (depending on their location) with near 100% probability.

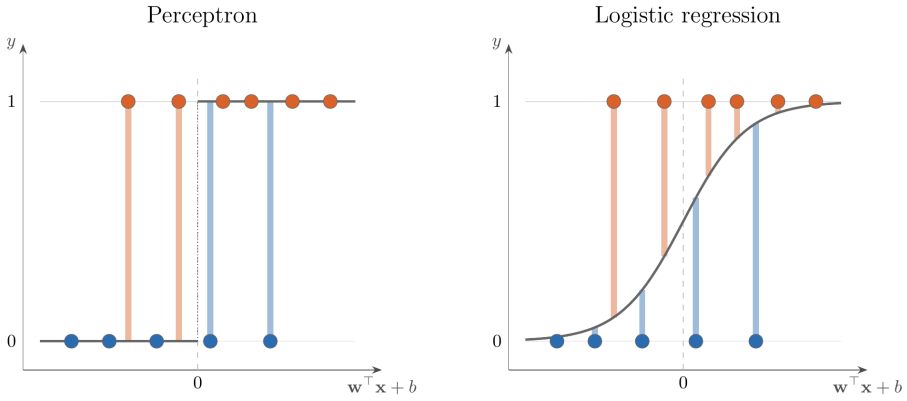


Figure 9: Class assignment in perceptron classification is fixed $\in \{0,1\}$, whereas is fixed in the range $\in [0,1]$ for logistic regression, allowing a probability of class membership to be defined.

Since we want to maximize the correct classification while minimizing the incorrect classifications, the objective function takes the form

$$J(\boldsymbol{w}) = \ln(\mathcal{L}) = \sum_{i=1}^{N} t_i \ln(y(\boldsymbol{x}_i)) + (1-t_i)\ln(1-y(\boldsymbol{x}_i)) \tag{85}$$

Since $y(\boldsymbol{x}_i) \equiv \phi(v)$ where $v = \boldsymbol{w}^T x_n + \boldsymbol{w}_0$, optimizing this objective function to find the ideal $\boldsymbol{w}$ requires applying the chain rule to bring out the explicit $\boldsymbol{w}$ dependence:

$$\frac{\partial J(\boldsymbol{w})}{\partial \boldsymbol{w}} = \sum_{i=1}^{N} t_i \frac{\partial \ln(\phi)}{\partial \phi}\frac{\partial \phi(v)}{\partial v}\frac{\partial v}{\partial \boldsymbol{w}} + (1-t_i)\frac{\partial \ln(1-\phi)}{\partial \phi}\frac{\partial \phi(v)}{\partial v}\frac{\partial v}{\partial \boldsymbol{w}} \tag{86}$$

Using the fact that

$$\frac{\partial \phi}{\partial v} = \phi(v)(1-\phi(v)) \quad \& \quad \frac{\partial v}{\partial \boldsymbol{w}} = \frac{\partial}{\partial \boldsymbol{w}}(\boldsymbol{w}^T \boldsymbol{x}_i + \boldsymbol{w}_0) = \boldsymbol{x}_i \tag{87}$$

$$\begin{aligned}\implies \frac{\partial J(\boldsymbol{w})}{\partial \boldsymbol{w}} &= \sum_{i=1}^{N}(1-\phi(v))t_i\boldsymbol{x}_i - (1-t_i)\phi(v)\boldsymbol{x}_i \\ &= \sum_{i=1}^{N}\boldsymbol{x}_i(t_i-\phi(v)) \equiv \sum_{i=1}^{N}\boldsymbol{x}_i(t_i-y(\boldsymbol{x}_i)).\end{aligned} \tag{88}$$

So the stochastic (per-sample) update of logistic regression is

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta\boldsymbol{x}_i(t_i - y(\boldsymbol{x}_i)) \tag{89}$$

and

$$\boldsymbol{w}_0^{(t+1)} = \boldsymbol{w}_0^{(t)} - \eta(t_i - y(\boldsymbol{x}_i)) \tag{90}$$

since the only thing that would change in the derivation for $\boldsymbol{w}_0$ would be the $\partial_{\boldsymbol{w}_0} v = 1$ instead of $\boldsymbol{x}_i$.

Notice that this is the same update equation as for the perceptron's $\boldsymbol{w}$ when $t_i - y(x_i)$ is either 0 or 1.

5.6 Support Vector Machines (SVM)

Another linear classifier, SVM builds on perceptron classification by again defining a linear discriminant function $\boldsymbol{w}^T\boldsymbol{x} + \boldsymbol{w}_0 = 0$, but seeks to draw this line where the gap between classes is as wide as possible. This helps choose an *optimal* discriminant function, whereas perceptron classification was just looking for *any* solution.

To construct the SVM objective function, first express each point $\boldsymbol{x}_n$ as being some distance from a given discriminant function line:

$$\boldsymbol{x}_n = \boldsymbol{x}_\perp + d\frac{\boldsymbol{w}}{\|\boldsymbol{w}\|} \tag{91}$$

where $\boldsymbol{w}_\perp$ is the projected position of $\boldsymbol{x}_n$ along the discriminant line and d is the Euclidean distance between $\boldsymbol{x}_\perp$ and $\boldsymbol{x}_n$ in the direction tangent to the discriminant function ($\hat{\boldsymbol{w}}$).

Since we want to maximize the distance between $\boldsymbol{x}_\perp$ and $\boldsymbol{w}_n$ (maximize d), we want an expression for $d(\boldsymbol{w})$ so that d is an expression we can maximize for an optimal $\boldsymbol{w}$. Left-multiplying $\boldsymbol{w}^T$ on both sides of the previous expression:

$$\boldsymbol{w}^T\boldsymbol{x}_n = \boldsymbol{w}^T\boldsymbol{x}_\perp + d\frac{\boldsymbol{w}^T\boldsymbol{w}}{\|\boldsymbol{w}\|} \tag{92}$$

and using the fact that $\boldsymbol{w}^T\boldsymbol{w} = \|\boldsymbol{w}\|^2$ and $\boldsymbol{x}_\perp$ exists *on* the discriminant function, so $\boldsymbol{w}^T\boldsymbol{x}_\perp + \boldsymbol{w}_0 = 0$ by definition, we get

$$d = \frac{\boldsymbol{w}^T\boldsymbol{x}_n + \boldsymbol{w}_0}{\|\boldsymbol{w}\|}. \tag{93}$$

Adding the t_n term to identify misclassifications $n \in \mathcal{M}$:

$$d = \frac{t_n(\boldsymbol{w}^T\boldsymbol{x}_n + \boldsymbol{w}_0)}{\|\boldsymbol{w}\|}. \tag{94}$$

Since we want to minimize the number of misclassifications,

$$J(\boldsymbol{w}) \propto \min_n \left[\frac{t_n(\boldsymbol{w}^T\boldsymbol{x}_n + \boldsymbol{w}_0)}{\|\boldsymbol{w}\|}\right]. \tag{95}$$

But since we *also* want the discriminant function that *maximizes* the class gaps, we also require

$$J(\boldsymbol{w}) = \max_{\boldsymbol{w},\boldsymbol{w}_0}\left[\min_n \left[\frac{t_n(\boldsymbol{w}^T\boldsymbol{x}_n + \boldsymbol{w}_0)}{\|\boldsymbol{w}\|}\right]\right] \tag{96}$$

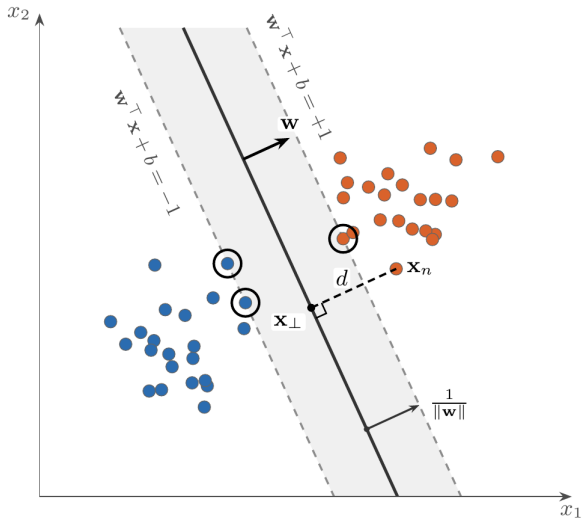


Figure 10: test.

6 Exercises

6.1 Discriminant Function Derivation

Problem 1: Find the discriminant function $g_1(\boldsymbol{x})$ for a 2 class *equal-prior* classification problem where

$$C_1 \sim \mathcal{N}(\mu_1, \Sigma_1) \quad \text{with} \quad \mu_1 = \begin{pmatrix} 3 \\ 6 \end{pmatrix} \quad \& \quad \Sigma_1 = \begin{pmatrix} 1/2 & 0 \\ 0 & 2 \end{pmatrix} \tag{97}$$

(find the function that defines where in $\boldsymbol{x} = (x_1, x_2)$ the model should define a point x_i to belong to class C_1).

This tells us we want to find the exact form of

$$g_1(\boldsymbol{x}) = \ln\big(P(C_1|x)\big) = \ln\left(\frac{P(x|C_1)P(C_1)}{P(x)}\right) \tag{98}$$

and we can drop the $P(x)$ term since this is just a normalization factor both discriminant functions $g_1(\boldsymbol{x})$ and $g_2(\boldsymbol{x})$ will contain, and we can drop the $P(C_1)$ term since this problem considers an equal-prior case, where samples are equally-likely to be pulled from either C_1 or C_2, meaning neither term will help us distinguish between the two classes, and we can drop their dependence in the discriminant function, leaving us with

$$g_1(\boldsymbol{x}) \sim \ln\big(P(x|C_1)\big) \tag{99}$$

Adding the 2-dimensional Gaussian form of the likelihood and plugging the given μ_1 and Σ_1 in:

$$\begin{aligned} g_1(\boldsymbol{x}) &\sim \ln\left(\frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right)\right) \\ &= -\frac{d}{2}\ln(2\pi) - \frac{1}{2}\ln(|\Sigma|) - \frac{1}{2}\left((x-\mu)^T\Sigma^{-1}(x-\mu)\right) \end{aligned} \tag{100}$$

$$|\Sigma| = 1 \quad \& \quad \Sigma^{-1} = \begin{pmatrix} 2 & 0 \\ 0 & 1/2 \end{pmatrix} \tag{101}$$

$$\begin{aligned} \implies g_1(\boldsymbol{x}) &\sim -\ln(2\pi) - \frac{1}{2}\left(\begin{pmatrix} x_1 - 3 & x_2 - 6 \end{pmatrix}\begin{pmatrix} 2 & 0 \\ 0 & 1/2 \end{pmatrix}\begin{pmatrix} x_1 - 3 \\ x_2 - 6 \end{pmatrix}\right) \\ &\sim -\ln(2\pi) - \frac{1}{2}\left(2(x_1-3)^2 + \frac{1}{2}(x_2-6)^2\right) = g_1(\boldsymbol{x}) \end{aligned} \tag{102}$$

Note that you would carry out the same method for $g_2(\boldsymbol{x})$ if you were given an exact form for the likelihood on C_2, $P(x|C_2)$.

6.2 Converting an Observed Data Likelihood into an Objective Function Expression

Problem 1: Convert the given observed data likelihood:

$$\mathcal{L} = \left(\prod_{i=1}^{N} \frac{1}{\sqrt{2\pi}} e^{-(t_i - y_i)^2/2}\right)\left(\prod_{j=1}^{M} \frac{1}{2b} e^{-|w_j|/b}\right) \tag{103}$$

into an optimization problem for linear regression.

In the language of observed data likelihoods, we want to maximize a posterior probability distribution to find the optimal model parameters, $\arg\max_W \mathcal{L}$, but in linear regression optimization, we want the model parameters which minimize the error of some objective function, $\arg\min_W J(W)$, so the task is to convert this from an $\arg\max \rightarrow \arg\min$ problem and transform $\mathcal{L} \rightarrow J(W)$.

To do this, we start with the observed data likelihood optimization expression and first get rid of some unnecessary scaling constants

$$\begin{aligned} \arg\max_W \mathcal{L} &\propto \arg\max_W \left[\left(\prod_{i=1}^{N} e^{-(t_i - y_i)^2/2}\right)\left(\prod_{j=1}^{M} e^{-|w_j|/b}\right)\right] \\ \implies \arg\max_W \ln \mathcal{L} &\propto \arg\max_W \left[-\sum_{i=1}^{N}(t_i - y_i)^2 - \sum_{j=1}^{M}\frac{|w_j|}{b}\right] \end{aligned} \tag{104}$$

And using the fact that $\arg\max(-f(x)) = \arg\min f(x)$, we get

$$\arg\max_W \mathcal{L} \propto \arg\max_W \left[\sum_{i=1}^{N}(t_i - y_i)^2 + \sum_{j=1}^{M}\frac{|w_j|}{b}\right] \tag{105}$$

And this is now in a form which can be written as the minimization of an objective function:

$$\arg\max_W \mathcal{L} \propto \arg\max_W \big(|\boldsymbol{t} - \boldsymbol{y}|^2 + \frac{1}{b}|\boldsymbol{W}|\big) \tag{106}$$

So we see that the $\exp(-|w_j|/b)$ term in the observed data likelihood is equivalent to adding a Lasso regularization to an MSE objective function.

6.3 Expectation Maximization Algorithm Practice

Problem 1: Consider a mixture model with 2 components ($K = 2$), where one component is Gaussian-distributed, $P_1(X|\theta) = \mathcal{N}(x|\mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp(-(x-\mu)^2/2\sigma^2)$, and the other component is Exponential-distributed, $P_2(X|\theta) = E(x|\lambda) = \lambda e^{-\lambda x}$.

Write the complete data likelihood by adding the latent variable z_i:

$$\begin{aligned}
\mathcal{L} &= \prod_{i=1}^{N} \prod_{k=1}^{K} \Big(\pi_k P_k(x|\theta_k) \Big)^{z_{ik}} \\
&= \prod_{i=1}^{N} \Big(\pi_1 P_1(x|\theta_1) \Big)^{z_{i1}} \Big(\pi_2 P_2(x|\theta_2) \Big)^{z_{i2}} \\
&= \prod_{i=1}^{N} \Big(\pi_1 \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2} \Big)^{z_{i1}} \Big(\pi_2 \lambda e^{-\lambda x} \Big)^{z_{i2}}
\end{aligned} \tag{107}$$

Now derive the update equation for the parameter λ:

Since we only care for the term that depends on λ, we can drop the objective function's dependence on the $k = 1$ Gaussian term:

$$\begin{aligned}
Q(\theta, \theta^{(t)}) &\propto \sum_i \gamma_{i2} \Big(\ln(\pi_2) + \ln\big(\lambda e^{-\lambda x_i}\big) \Big) \\
\implies \frac{\partial Q(\theta, \theta^{(t)})}{\partial \lambda} &\propto \frac{\partial}{\partial \lambda} \Big(\sum_i \gamma_{i2} \ln\big(\lambda e^{-\lambda x_i}\big) \Big) = 0 \\
&= \frac{\partial}{\partial \lambda} \Big(\sum_i \gamma_{i2} \big(\ln \lambda - \lambda x_i \big) \Big) \\
&= \sum_i \gamma_{i2} \Big(\frac{1}{\lambda} - x_i \Big) = 0 \\
\implies \lambda &= \frac{\sum_i \gamma_{i2}}{\sum_i \gamma_{i2} x_i}
\end{aligned} \tag{108}$$

7 To Know Cold

1. Objective function for mean-squared error linear regression ($\boldsymbol{y} = \boldsymbol{X}\boldsymbol{w}$ where $\boldsymbol{X}\boldsymbol{w} \equiv \boldsymbol{w}^T\phi(\boldsymbol{x})$ and $\phi(\boldsymbol{x}) = [\phi_1(\boldsymbol{x}), \ldots, \phi_M(\boldsymbol{x})]^T$):

$$J(W) = \sum_{i=1}^{N} \frac{(t_i - y_i)^2}{N} \equiv |\boldsymbol{t} - \boldsymbol{X}\boldsymbol{w}|^2 \tag{109}$$

   With optimal weights found by

$$\left.\frac{\partial J}{\partial W}\right|_{W^*} = 0 \tag{110}$$

   and corresponding optimal solution

$$W^* = (X^T X)^{-1} X^T t \tag{111}$$

2. Derivatives acting on transposed variables:

$$\frac{\partial}{\partial W}\left(W^T X^T X W\right) = (X^T X W)^T + W^T X^T X = 2W^T X^T X \tag{112}$$

3. MLE Optimization Algorithm:

$$\left.\frac{\partial \mathcal{L}_{\text{MLE}}}{\partial \theta}\right|_{\theta^*} = 0 \quad \text{where} \quad \mathcal{L}_{\text{MLE}} = \prod_{i=1}^{N} P(x_i|\theta) \tag{113}$$

4. MAP Optimization Algorithm:

$$\left.\frac{\partial \mathcal{L}_{\text{MAP}}}{\partial \theta}\right|_{\theta^*} = 0 \quad \text{where} \quad \mathcal{L}_{\text{MAP}} = \left(\prod_{i=1}^{N} P(x_i|\theta)\right) P(\theta) \tag{114}$$

5. Product Rules

$$\begin{aligned} \prod_{i=1}^{N} p(1-p)^{x_i - 1} &= p^N (1-p)^{\sum_i x_i - N} \\ \prod_{i=1}^{N} \lambda e^{-\lambda x_i} &= \lambda^N e^{-\lambda \sum_i x_i} \end{aligned} \tag{115}$$

6. 2x2 matrix inverse

$$A^{-1} = \frac{1}{|A|} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \tag{116}$$

where $|A| = ad - bc$

7. Mixtures of various distributions:

   - Exponentials:

$$P(X|\theta) = \prod_{i=1}^{N} \sum_{k=1}^{K} \pi_k \lambda_k e^{-\lambda_k x_i} \tag{117}$$

   - Gaussians:

$$P(X|\theta) = \prod_{i=1}^{N} \sum_{k=1}^{K} \pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k) \tag{118}$$

   - Bernoullis:

$$P(X|\theta) = \prod_{i=1}^{N} \sum_{k=1}^{K} \pi_k p_k^{x_i} (1 - p_k)^{1-x_i} \tag{119}$$

     for binary data $x_i \in \{0, 1\}$

   - Poissons:

$$\prod_{i=1}^{N} \sum_{k=1}^{K} \pi_k \frac{\lambda_k^{x_i} e^{-\lambda_k}}{x_i!} \tag{120}$$

     for sequences $x_i \in \{0, 1, 2, \dots\}$

8. Naive Bayes Classifier (for two class classification):

$$g(X) = g_1(X) - g_0(X) \tag{121}$$

where

$$g_1(X) \propto \ln\big(P(X|C_1)\big) \neq^{!!!} \ln\left(\prod_{i=1}^{N} P(x_i|C_1)\right) \tag{122}$$

9. Second Parameter update example for $X = \{x_i\}$ and $N^{(t+1)} = |X^{(t+1)}| - |X^{(t)}|$

$$\theta_*^{(t+1)} = \frac{N^{(t+1)} + \alpha^{(t+1)}}{\sum_{i \in X^{(t+1)}/X^t} x_i + \beta^{(t+1)}} \tag{123}$$

10. 4 Methods for preventing overfitting:

    (a) Regularization:

$$J'(W) = J(W) + \lambda\Omega(W) \tag{124}$$

    where $\Omega(W)$ can be

    i. L_2/Ridge:

$$\Omega(W) = \sum_j w_j^2 \tag{125}$$

    ii. L_1/Lasso:

$$\Omega(W) = \sum_j |w_j| \tag{126}$$

    iii. Elastic Net:

$$\Omega(W) = \alpha \sum_j |w_j| + (1-\alpha) \sum_j w_j^2 \tag{127}$$

    (b) Occam's Razor (choose simpler model$\equiv$ model with fewer necessary parameters)

    (c) Cross-Validation (ex: $K-$fold)

    (d) Addition of more data

11. Observed data likelihood optimization $\leftrightarrow$ objective function optimization

$$\arg\max_W \mathcal{L} = \arg\min_W J(W) \tag{128}$$

12. Conversion from $\arg\max_W \leftrightarrow \arg\min_W$:

$$\arg\max(-f(x)) = -\arg\max(f(x)) = \arg\min(f(x)) \tag{129}$$

13. Expectation Maximization Algorithm:

    (a) Expectation (E) Step:

$$Q(\theta, \theta^{(t)}) = \mathbb{E}_Z\left[\ln \mathcal{L} \middle| X, \theta\right] \tag{130}$$

    where

$$\mathcal{L} = \prod_{i=1}^{N} \prod_{k=1}^{K} \Big(\pi_k P_k(x_i|\theta_k)\Big)^{z_{ik}} \tag{131}$$

for example, for Gaussian data likelihoods, $P_k(x_i|\theta_k) = \mathcal{N}(x_i|\mu_k, \sigma_k)$. This results in

$$Q(\theta, \theta^{(t)}) = \sum_{i=1}^{N} \sum_{k=1}^{K} \gamma_{ik} \left(\ln(\pi_k) + \ln\left(P_k(x_i|\theta_k) \right) \right) \tag{132}$$

where

$$\gamma_{ik}^{(t)} \equiv P_k(z_{ik} = 1|x_i, \theta^{(t)}) = \frac{P_k(x_i|\theta_k^{(t)})\pi_k^{(t)}}{\sum_{j=1}^{K} P_j(x_i|\theta_k^{(t)})\pi_j^{(t)}} \tag{133}$$

(b) Maximization (M) Step:

$$\theta^{(t+1)} = \arg\max_{\theta} Q(\theta, \theta^{(t)}) \equiv \left. \frac{\partial Q(\theta, \theta^{(t)})}{\partial \theta} \right|_{\theta=\theta^{(t+1)}} = 0 \tag{134}$$

14. Bayes Theorem

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \tag{135}$$